NextGen QA

AI-Driven Framework for End-to-End Software Testing Lifecycle Automation



Project ID: R26-SE-039

Project Title: NextGen QA – AI-Driven Framework for End-to-End Software Testing Lifecycle Automation

Student Details:

Student ID	Student Name
IT22251114	Jinad Induvidwa A.G
IT22303684	Abeygunasekara D.T
IT22371768	Ranasinghe S.I
IT22182678	Rashani K.G.M

Supervisor: Ms. Suriyaa Kumari

Co-Supervisor: Mr. Eishan Weerasinghe

Date of Submission: 24–09–2026

Contents

1. Executive Summary and Objectives 4
   Deployment Overview 4
   Purpose and Outcomes 5
      Purpose: 5
      Outcomes: 5
   Key Metrics 6
   Goals and Scope 7
      Goals: 7
      Scope: 7
2. Deployment Process 9
   Pre-Deployment Setup & Configuration 9
      Environment Preparation 9
      Configuration of Environment Variables 9
      Service Configuration and Port Allocation 9
      API and Integration Setup 10
      Client Configuration 10
      Connectivity and Network Setup 10
      Dependency and Build Validation 10
   Testing Activities (Unit, Integration, UAT) 11
      1. Unit Testing 11
      2. Integration Testing 11
      3. User Acceptance Testing (UAT) 12
      4. Overall Testing Summary 12
   Backup Plan and Rollback Strategy 13
      Backup Plan 13
      Rollback Strategy 13
      Risk Mitigation Strategy 14
      Outcome 14
   Deployment Details (Date, Time, Team Members, Tools Used) 14
      Deployment Information 14
      Team Members Involved 14
      Tools and Technologies Used 15
   Deployment Checklist (Pre-, During- and Post-Deployment Tasks) 15
      Pre-Deployment Checklist 15
      During Deployment Checklist 16
      Post-Deployment Checklist 16
3. Issues, Resolutions and Monitoring 18
   List all issues 18

Resolutions and Mitigation 18
Impact Analysis of Issues 19
Monitoring, Performance Metrics, and Feedback 19
4. Post-Deployment and Lessons Learned 20
Validation and Testing Results After Deployment 20
Documentation Updates and Training / Support Activities 21
Key Lessons Learned 21
Recommended Future Improvements 22
Application Demo Video 22

1. Executive Summary and Objectives

Deployment Overview

The deployment of the NextGen QA system focused on making the complete AI-driven software testing lifecycle framework operational as a single, cloud-hosted, integrated platform within a development and user acceptance testing (UAT) environment.

NextGen QA is designed as a multi-component, service-oriented system that integrates a web client, an API gateway, an authentication service, several Python-based AI microservices, and a set of per-service databases. Each research component of the project was developed independently by a team member, and the goal of this deployment was to ensure that all of these components work together as one coherent, human-governed workflow supporting the full testing lifecycle from requirement capture to traceable test coverage.

The deployed system consists of the following main components:

- **Client Application –** A React single-page web application (React 19, Vite and TailwindCSS) hosted on Vercel, used by Business Analysts, QA Engineers, Developers and QA Leads to interact with every service through a shared authenticated interface
- **Edge and Gateway –** A Caddy reverse proxy providing automatic HTTPS, and a Node.js/Express API gateway responsible for JWT authentication and routing requests to the correct backend service
- **Authentication Service –** The single sign-on service that issues tokens and acts as the system of record for user identity and authorization
- **Intelligent User Story Generation Service (Component 1) –** Real-time meeting transcription and speaker identification, requirement classification, requirement threading, conflict detection, human-in-the-loop resolution and AI-based user story generation with validation
- **Test Case Generation Service (Component 2) –** Gherkin scenario generation, live application inspection, a vision-guided Agent Explorer, multi-framework test code generation (Selenium, Playwright and Cypress) and CI execution with reporting
- **Self-Healing Service (Component 3) –** Machine-learning root-cause classification of CI/CD failures, a confidence gate, a deterministic recovery policy and a safe two-phase repair workflow with developer approval
- **RTM Generation Service (Component 4) –** Test inventory, quality scoring, coverage-gap detection, risk-based prioritization, live requirements traceability matrix (RTM) updates and an intelligent test improvement engine
- **Data Layer –** Managed PostgreSQL on Neon with one isolated database per service, and vector search enabled for semantic retrieval in Component 1
- **External Integrations –** Azure Speech for transcription, Claude, OpenAI, Gemini and OpenRouter for AI reasoning and generation, and GitHub for CI runners and controlled repository operations

All backend services run as Docker containers on a single AWS EC2 host, with one container per service. Only the reverse proxy is exposed to the public internet; every other service is reachable only on the internal container network.

The deployment enabled the following core system workflows:

- **Requirement Harmonization Flow –** Meeting audio is transcribed, requirement-bearing utterances are classified and threaded, conflicts are detected and resolved by a Business Analyst, and a validated user story is generated with a quality score
- **Test Generation Flow –** A validated user story is converted into approved Gherkin, grounded against the live application, generated as executable test code and executed on a clean GitHub Actions runner with live logs and an Allure report

- **Self-Healing Flow –** A failed test execution is classified into a root cause, routed through a confidence gate and recovery policy, and only an approved application defect produces an isolated repair branch and draft pull request
- **Traceability and Coverage Flow –** Real test execution evidence drives test quality scoring, coverage-gap detection, risk prioritization and live RTM status, with low-quality tests routed back into regeneration

This deployment ensured that all components were:

- Fully integrated and communicating through a single authenticated gateway
- Functionally complete for demonstration purposes (≈90–95%)
- Tested under realistic end-to-end usage scenarios
- Stable enough for live demonstration and final evaluation

Overall, the deployment successfully transformed the NextGen QA system from four independently developed research prototypes into a cohesive, working platform, demonstrating both system maturity and research maturity.

Purpose and Outcomes

Purpose:

The primary purpose of deploying the NextGen QA system was to operationalize the complete AI-driven testing lifecycle framework as a fully integrated and functional system within a controlled cloud environment.

Specifically, the deployment aimed to:

- Enable end-to-end workflows, including requirement capture from meetings, user story generation, test case generation, CI execution, failure analysis, self-healing and traceability reporting
- Validate the integration between all system components, including the web client, edge proxy, API gateway, authentication service, the four research microservices and the repair agent
- Ensure reliable communication between the Node.js gateway layer and the Python-based AI microservices
- Confirm that real-time channels such as WebSocket streams for meeting audio and execution logs, and Server-Sent Events for inspector logs, work correctly through the reverse proxy and gateway
- Preserve the human-governance principle of the framework, where AI components analyze and propose but consequential actions pass through explicit approval gates
- Verify that the system meets the expected functional and performance requirements

Outcomes:

The deployment successfully achieved the following outcomes:

- **Full system integration**
  - All five microservices and the repair agent were connected behind one API gateway and one public HTTPS endpoint, and were operational as a single platform
- **End-to-end workflow validation**
  - The system supported the complete lifecycle from meeting transcription through user story approval, test generation, CI execution, controlled self-healing and live RTM status
- **Stable service communication**
  - REST, WebSocket and Server-Sent Events traffic passed correctly through the proxy chain after CORS, upgrade-header and response-buffering configuration was corrected

- **Safety behaviour preserved**
    - The confidence gate, deterministic recovery policy, approval gate and repository access controls behaved exactly as designed in the deployed environment, with no unsafe repository modification observed
- **Successful testing and demonstration readiness**
    - The system was tested under multiple scenarios and is stable enough for live demonstration
- **Issue identification and resolution**
    - Deployment-related issues such as disk space, database SSL requirements, CORS errors, certificate issuance, streaming through the proxy and credential scope were identified and resolved
- **Improved system reliability and maturity**
    - The deployment confirmed that the system has reached approximately 90–95% completion, indicating strong readiness for evaluation and presentation

Key Metrics

To evaluate the effectiveness of the NextGen QA system after deployment, several key performance indicators were considered. These reflect the system's reliability, accuracy and overall behaviour when running as an integrated platform. Values are reported as approximate observed results from testing on the deployed system.

1. Requirement Classification Accuracy

- **Definition:** Ability of Component 1 to correctly classify meeting utterances into requirement categories
- **Result:** ~94% (macro-averaged across nine categories)

2. Generated Test Executability

- **Definition:** Percentage of generated test cases that execute successfully on first run against the target application
- **Result:** ~91% when grounded against the live application, compared with ~79% for ungrounded generation

3. Root-Cause Classification Accuracy

- **Definition:** Ability of Component 3 to correctly identify the cause of a CI/CD test failure
- **Result:** ~89% (macro-averaged across nine failure categories)

4. Unsafe Repository Modification Rate

- **Definition:** Frequency of any repository change made without developer approval or outside the permitted scope
- **Result:** 0%, with 100% approval-gate enforcement

5. Traceability Update Latency

- **Definition:** Time taken for the RTM status to be recalculated after a relevant test or coverage event
- **Result:** ~150 milliseconds per update

6. Service Availability (Uptime)

- **Definition:** Percentage of time all system services reported healthy during testing and demonstration
- **Result:** High availability after stabilization (>95%)

7. Error Rate

- **Definition:** Frequency of critical errors affecting system functionality
- **Result:** Low after deployment stabilization

Goals and Scope

Goals:

The primary goal of this deployment was to ensure that the NextGen QA system operates as a fully integrated and functional solution, capable of supporting the complete Agile software testing lifecycle from requirement capture through to execution-verified test coverage.

The specific goals include:

- Achieve full integration of all system components, including the web client, edge proxy, API gateway, authentication service, AI microservices and databases
- Enable end-to-end workflows, such as requirement harmonization, test generation, CI execution, failure classification, controlled self-healing and traceability reporting
- Ensure stable communication between services, especially between the Node.js gateway and the Python-based AI microservices
- Preserve every human-approval gate, safety policy and access-control boundary in the deployed configuration
- Validate that the system is functionally complete (approximately 90–95%) and ready for demonstration
- Demonstrate the application of AI techniques such as contextual classification, semantic conflict detection, vision-guided exploration, root-cause classification and quality scoring
- Provide a reliable and scalable foundation for future production deployment

Scope:

Included in Scope:

The deployment covered the following components and functionalities:

- **Client Application**
  - React single-page application hosted on Vercel, covering the interfaces of all four research components behind a shared authenticated shell
- **Edge and Gateway**
  - Caddy reverse proxy with automatic HTTPS
  - Node.js/Express API gateway with JWT authentication and prefix routing
- **Backend Services**
  - Authentication Service for identity and single sign-on
  - Intelligent User Story Generation Service
  - Test Case Generation Service
  - Self-Healing Service together with the isolated repair agent
  - RTM Generation Service
- **Data Layer**
  - Managed PostgreSQL on Neon with one database per service
  - Vector search extension for semantic retrieval in Component 1
- **Core Workflows**
  - Meeting capture, requirement classification, conflict resolution and user story approval
  - Gherkin approval, application grounding, test code generation and CI execution
  - Failure submission, root-cause classification, repair proposal and developer approval
  - Test inventory, quality scoring, coverage analysis, RTM browsing and recommendation review

Excluded from Scope:

The following aspects were not included in this deployment phase:

- Multi-node or auto-scaling production infrastructure; the platform runs on a single cloud host
- Centralized monitoring and logging dashboards beyond the health endpoint and container logs
- A CI/CD pipeline for deploying the NextGen QA platform itself
- Container or virtual-machine sandboxing for the coverage agent when analysing external repositories
- Advanced security hardening, penetration testing and formal compliance measures

2. Deployment Process

Pre-Deployment Setup & Configuration

Before deploying the NextGen QA system, several preparation and configuration activities were carried out to ensure that all components could function correctly within the cloud deployment environment.

Environment Preparation

The deployment environment was prepared by provisioning and verifying all required infrastructure, software and dependencies:

- An AWS EC2 instance in the Singapore region with a static IP address as the platform host
- Docker Engine and Docker Compose for one-container-per-service orchestration
- Node.js and npm for the API gateway and Authentication Service
- Python environments for the four FastAPI research services and the repair agent
- Chromium and Playwright browser dependencies required by the Test Case Generation Service
- Managed PostgreSQL on Neon with one database created per service
- A public hostname pointed at the server's static IP address

Configuration of Environment Variables

All necessary environment variables were defined and verified to ensure proper communication between services and with external providers. Secrets are held only on the server side and are never sent to the web client.

- Database connection strings for each service, one isolated database per service
- JWT signing and verification keys shared between the Authentication Service, the gateway and all protected services
- Azure Speech credentials for real-time transcription and speaker identification
- API keys for Gemini, Claude, OpenAI and OpenRouter used for embeddings, generation, exploration and repair reasoning
- A read-only GitHub token for reading workflow runs and logs, and a separately scoped write token used only by the repair agent
- The repair confidence threshold and the list of protected file paths that may never be changed
- Allowed CORS origins for the gateway and the backend service URLs used for routing

These variables were carefully documented and managed to avoid configuration mismatches during deployment.

Service Configuration and Port Allocation

Each component of the system was assigned a dedicated port to avoid conflicts and ensure smooth communication. Only the reverse proxy binds to the public network interface; every other port is reachable only on the internal Docker network.

- Caddy reverse proxy – Ports 80 and 443 (public)
- API gateway and Authentication Service – Internal ports on the Docker network
- Intelligent User Story Generation Service – Port 8001
- Test Case Generation Service – Port 8002
- Self-Healing Service – Port 8000
- Repair Agent – Port 8010
- RTM Generation Service – Port 8003

- Managed PostgreSQL – Port 5432 over TLS (external managed service)

Port allocation was validated to ensure no conflicts with other running services on the host.

API and Integration Setup

- Gateway prefix routes were configured for every backend service and verified against the aggregate health endpoint
- JWT verification middleware was enabled on the gateway so that one Authentication Service token is trusted by every protected service
- CORS settings were configured to allow requests from the Vercel web client origin
- Service-to-service APIs were configured so that the Self-Healing Service obtains failed-execution evidence from the Test Case Generation Service without direct database access
- WebSocket upgrade paths were configured through the proxy and gateway for meeting audio streaming, agent events and execution logs
- Server-Sent Events pass-through was configured for inspector logs with response buffering disabled, so log lines arrive incrementally
- API endpoints were tested using tools such as Postman and Thunder Client

Client Configuration

- The web application was configured with the production API base URL and deployed to Vercel
- Token storage, protected routes and the shared authenticated shell were verified across all four service interfaces
- Live streaming panels were verified end to end, including the Agent Explorer view, the execution terminal and the coverage log panel
- Basic UI flows were verified for meeting capture, conflict resolution, story approval, Gherkin editing, code review, execution, failure submission, repair approval, RTM browsing and recommendation review

Connectivity and Network Setup

- The server security group was configured to allow inbound traffic on ports 80 and 443 only
- All service ports were bound to the internal Docker network rather than the host interface
- Port 80 reachability was confirmed so that the automatic HTTPS certificate challenge could complete
- Outbound connectivity was confirmed to the managed database, Azure Speech, the AI providers and GitHub
- DNS resolution of the public hostname was confirmed from an external network before the first certificate request

Dependency and Build Validation

- All service container images were built and build failures were resolved before first start
- Playwright browser dependencies were installed inside the Test Case Generation image and a headless browser launch was verified
- Trained model files were confirmed present for the requirement classifier, the root-cause classifier and the quality scoring model
- Services were started individually to confirm successful initialization before full composition
- Initial health checks were performed to ensure all services were operational

Testing Activities (Unit, Integration, UAT)

To ensure the reliability, stability and correctness of the NextGen QA system, multiple levels of testing were conducted during and after deployment. These include unit testing, integration testing and user acceptance testing (UAT).

1. Unit Testing

Unit testing was performed on individual system components to verify that each module functions correctly in isolation.

Activities:

- Tested Authentication Service token issuance and expiry, and gateway rejection of unsigned or expired tokens
- Validated Component 1 independently: transcription handling, contextual utterance classification, semantic retrieval, requirement thread transitions, conflict detection and the user story validation stages
- Validated Component 2 independently: Gherkin templates, the deterministic selector builder, visual element marking for the Agent Explorer, state tracking for exploration termination and the per-framework code generators
- Validated Component 3 independently: root-cause inference with confidence output, the confidence gate, the recovery policy mapping, eligibility checks and protected-path rejection
- Validated Component 4 independently: the weighted quality characteristics, the approval-eligibility threshold, the RTM status states, the risk classifier and each per-language coverage runner
- Verified database operations per service, confirming that no service reads another service's database

Outcome:

- All core services were able to start successfully and respond correctly in isolation
- Minor issues such as configuration errors, input-format mismatches and missing environment variables were identified and fixed

2. Integration Testing

Integration testing was conducted to ensure that all system components work together as a fully integrated solution.

Activities:

- Tested communication between:
    - Web client ↔ Reverse proxy ↔ API gateway ↔ each backend service over REST, WebSocket and Server-Sent Events
    - API gateway ↔ Authentication Service for single sign-on
    - Each service ↔ its own database
    - Each service ↔ its external provider (Azure Speech, AI providers and GitHub)
- Verified data flow across modules:
    - Approved user story → test generation → CI execution → failure classification → traceability update
    - Approved improvement recommendation → regeneration request back into test generation
- Tested API communication using tools such as Postman and Thunder Client
- Validated CORS configuration, WebSocket upgrade handling and streaming behaviour through the full proxy chain

Outcome:

- All modules were successfully integrated and communicating through the authenticated gateway
- Issues such as certificate issuance, CORS errors, buffered streams, port binding and credential scope were identified and resolved
- End-to-end workflows across the full lifecycle were confirmed to be operational

3. User Acceptance Testing (UAT)

User Acceptance Testing was conducted to validate whether the system meets the intended user requirements and real-world use cases. Scenarios were organized by the role that performs them.

Activities:

Simulated real user scenarios:

- **Business Analyst Flow**
  - Joining a live meeting, watching utterances classified in real time, reviewing and resolving a detected conflict, and approving the generated user story with its quality score and acceptance criteria
- **QA Engineer Flow**
  - Importing an approved story, reviewing and approving Gherkin, choosing the generation mode and framework, approving the captured element map, reviewing generated code and executing the suite on GitHub Actions with live logs and an Allure report
- **Autonomous Exploration Flow**
  - Providing one intent sentence and a URL, watching the Agent Explorer run, and confirming that discovered scenarios are returned as Gherkin subject to the same human approval
- **Developer Flow**
  - Submitting a failed execution, reviewing the predicted root cause and confidence, reviewing the read-only repair proposal, withholding approval and confirming nothing is created, then approving and confirming exactly one repair branch, one commit and one draft pull request
- **QA Lead Flow**
  - Browsing the test inventory, reviewing quality scores, working the ranked coverage-gap list, running the coverage agent on a permitted repository, reviewing RTM status and accepting or rejecting an improvement recommendation

Outcome:

- All major workflows were successfully executed by the intended roles in test scenarios
- Users were able to:
  - Capture requirements from a meeting and approve a generated user story
  - Generate, review and execute test suites
  - Classify failures and complete a controlled repair
  - Review traceability, coverage gaps and improvement recommendations
- The safety-critical behaviours, including the confidence gate, policy routing, approval gate and access control, behaved as specified

4. Overall Testing Summary

- The system achieved functional completeness (~90–95%) across all five microservices
- All critical components and integration surfaces were tested and validated
- The system demonstrated stable performance under controlled testing conditions
- The platform is ready for live demonstration and final evaluation

Backup Plan and Rollback Strategy

To ensure system reliability and minimize risks during deployment, a structured backup and rollback strategy was implemented for the NextGen QA system.

Backup Plan

The backup plan focuses on preserving critical system configurations, code, model files and data to allow recovery in case of failures.

Key Backup Measures:

- **Configuration Backup**
  - Environment variables (API keys, database URLs, service endpoints, port assignments) were documented and stored securely
  - A working configuration snapshot was maintained before applying deployment changes
- **Codebase Backup**
  - Each service was maintained in a version control system (GitHub)
  - A release tag and container image tag were created before deployment updates
- **Database Backup**
  - Each per-service database was backed up before schema or data changes using the managed provider's branching and point-in-time restore features together with a data export
- **Model File Backup**
  - The requirement classifier, root-cause classifier and quality scoring model were versioned and stored outside the container images
- **Demo Backup**
  - Pre-recorded demo videos and screenshots were prepared for every service
  - Sample test data was stored to quickly re-run scenarios if needed

Rollback Strategy

The rollback strategy defines how the system can be restored to a previous stable state in case of deployment failure.

Rollback Actions:

- **Container-Level Rollback**
  - Redeploy the previous image tag for the affected service while leaving the other services running
- **Code Rollback**
  - Revert to the last stable Git tag and rebuild only the affected image
  - Undo recent changes that caused failures
- **Configuration Rollback**
  - Restore previous environment variable settings
  - Reapply known working API keys, service URLs and port mappings
- **Database Rollback**
  - Restore the affected service's database branch or export if data corruption or inconsistencies occur
  - Avoid risky schema changes during the deployment window
- **Partial Rollback**
  - Temporarily disable non-critical features such as the Agent Explorer, the repair publishing phase or the coverage agent

- Keep core functions such as story generation, test generation, failure classification and RTM viewing active

Risk Mitigation Strategy

To reduce deployment risks:

- Services were started in a controlled sequence (database → Authentication Service → gateway → research services → repair agent → reverse proxy → web client)
- Each component was tested individually before full integration, with health checks gating each stage
- The GitHub write credential remained restricted to the allow-listed repository throughout, so no rollback scenario could widen its impact
- Errors were logged and resolved step-by-step, service by service, rather than by restarting the whole system
- Backup resources ensured quick recovery without affecting the demonstration

Outcome

- No major system failure occurred after stabilization
- All critical issues were resolved using controlled, service-scoped rollback steps
- The system remained stable and ready for demonstration

Deployment Details (Date, Time, Team Members, Tools Used)

Deployment Information

Project Name: NextGen QA – AI-Driven Framework for End-to-End Software Testing Lifecycle Automation

Project ID: R26-SE-039

Deployment Environment: AWS EC2 cloud host with Vercel client hosting and managed PostgreSQL on Neon – development and UAT

Deployment Type: Cloud, multi-service container deployment (Docker Compose, one container per service)

Architecture: Five independently deployable microservices with one database per service and API-only communication

Deployment Date: 28 – 08 – 2026

Deployment Start Time: 09:30 (UTC+5:30)

Deployment End Time: 11:15 (UTC+5:30)

Deployment Lead: Abeygunasekara D.T

Team Members Involved

- **Jinad Induvidwa A.G (IT22251114) –** Intelligent User Story Generation: real-time speech processing, contextual requirement classification, threading, conflict detection and AI-based story generation with validation
- **Abeygunasekara D.T (IT22303684) –** Test Case Generation: Gherkin and code generation pipeline, application inspection, vision-guided Agent Explorer, multi-framework code generation and CI execution
- **Ranasinghe S.I (IT22371768) –** Self-Healing: root-cause classifier, confidence gate, deterministic recovery policy, two-phase repair workflow and audit logging
- **Rashani K.G.M (IT22182678) –** RTM Generation: test inventory, quality scoring, coverage-gap detection, risk prioritization, live RTM and recommendation engine

Tools and Technologies Used

Development Tools

- Visual Studio Code (VS Code)
- GitHub (version control, Actions CI runners and controlled repository tooling)
- Docker and Docker Compose
- Terminal / SSH access to the cloud host

Infrastructure and Edge

- AWS EC2 (Singapore region)
- Caddy reverse proxy with automatic HTTPS
- Vercel (web client hosting)

Backend Technologies

- Node.js with Express and TypeScript (API gateway and Authentication Service)
- Python with FastAPI (research services and repair agent)
- Managed PostgreSQL on Neon with vector search support

Frontend Technologies

- React 19, Vite and TailwindCSS single-page application

AI, ML and Automation

- Azure Speech SDK for real-time transcription and speaker identification
- Fine-tuned transformer classifier for contextual utterance classification
- Vision language model with visual element marking for agentic UI exploration
- Claude, OpenAI, Gemini and OpenRouter as generation, exploration and repair-reasoning back-ends
- scikit-learn for the root-cause classifier and the quality scoring model

Testing Tools

- Postman / Thunder Client (API testing)
- GitHub Actions for clean-runner execution of generated suites
- Browser developer tools for WebSocket, streaming and CORS verification

Deployment Checklist (Pre-, During- and Post-Deployment Tasks)

Pre-Deployment Checklist

- Provision the cloud host and attach a static IP address — **✓ Completed**
- Point the public hostname at the host IP and confirm external DNS resolution — **✓ Completed**
- Install and verify Docker Engine and Docker Compose on the host — **✓ Completed**
- Install Chromium and Playwright browser dependencies for the Test Case Generation Service — **✓ Completed**
- Create one database per service and record every connection string — **✓ Completed**
- Validate database connectivity from the host over TLS for every service — **✓ Completed**
- Configure and verify all environment variables (DB URIs, API keys, service URLs) — **✓ Completed**
- Confirm the GitHub write token is scoped to the allow-listed repository only — **✓ Completed**
- Check external services (Azure Speech, Claude, Gemini, OpenAI, OpenRouter, GitHub) — **✓ Completed**
- Confirm trained model files are present for all three ML components — **✓ Completed**

- Assign and verify all service ports and confirm no host-level conflicts — ✓ **Completed**
- Configure the reverse proxy for automatic HTTPS and confirm port 80 is reachable — ✓ **Completed**
- Configure gateway routing, JWT verification and CORS for the web client origin — ✓ **Completed**
- Build every service image and resolve all build failures — ✓ **Completed**
- Deploy the web application to Vercel with the correct API base URL — ✓ **Completed**
- Prepare test data and demo scenarios for every service — ✓ **Completed**
- Take a pre-deployment backup of configuration, tagged code, model files and every database — ✓ **Completed**

During Deployment Checklist

- Confirm all databases are reachable from the host — ✓ **Completed**
- Start the Authentication Service and verify token issuance — ✓ **Completed**
- Start the API gateway and verify authentication and routing — ✓ **Completed**
- Start the four research services in sequence and verify each starts without runtime errors — ✓ **Completed**
- Start the reverse proxy and confirm a valid HTTPS certificate is issued — ✓ **Completed**
- Verify every service reports healthy at the aggregate health endpoint — ✓ **Completed**
- Load the web application over HTTPS and complete a single sign-on login end to end — ✓ **Completed**
- Verify WebSocket streams for meeting audio, agent events and execution logs — ✓ **Completed**
- Execute the Component 1 flow: meeting capture → classification → conflict → resolution → validated story — ✓ **Completed**
- Execute the Component 2 flow: story → Gherkin → grounding → code generation → CI run → report — ✓ **Completed**
- Execute the Component 3 flow: failed execution → classification → proposal → approval → draft pull request — ✓ **Completed**
- Verify every non-application root cause receives its recovery action with no repository modification — ✓ **Completed**
- Execute the Component 4 flow: inventory → quality score → coverage gaps → risk → live RTM → recommendation — ✓ **Completed**
- Monitor logs and identify errors during each flow — ✓ **Completed**
- Fix issues (disk space, database SSL, CORS, network and configuration issues) — ✓ **Completed**

Post-Deployment Checklist

- Validate all end-to-end user workflows (Business Analyst, QA Engineer, Developer, QA Lead) from the deployed client — ✓ **Completed**
- Confirm system integration across all modules and every cross-service hand-off — ✓ **Completed**
- Confirm no service accesses another service's database directly — ✓ **Completed**
- Confirm the approval gate blocks all publishing when approval is withheld — ✓ **Completed**
- Confirm no repository other than the allow-listed one can be modified — ✓ **Completed**
- Ensure no critical errors remain in any container log — ✓ **Completed**

- Measure system performance (response time, accuracy, availability) **— ✓ Completed**
- Record deployment results and observations **— ✓ Completed**
- Document issues and resolutions **— ✓ Completed**
- Prepare backup demo (screenshots/videos) **— ✓ Completed**
- Final system stability verification **— ✓ Completed**

3. Issues, Resolutions and Monitoring

List all issues

During the deployment of the NextGen QA system, several technical and integration-related issues were identified:

- **Insufficient Disk Space**
  - Container image pulls failed on the cloud host during the deployment run because leftover images from earlier builds had consumed the available disk
- **Database SSL Requirement**
  - The Test Case Generation Service could not connect to the managed database because the connection string did not request an SSL connection
- **CORS (Cross-Origin Resource Sharing) Errors**
  - The web client hosted on Vercel was blocked when calling the backend API because its origin was not yet in the gateway's allowed list
- **Streaming Through the Proxy**
  - Live inspector and agent logs arrived only at the end of a run until WebSocket upgrade handling was enabled and response buffering was disabled
- **Service Integration Issues**
  - Incorrect service URLs and inconsistent data formats caused communication failures between the gateway and the Python services
- **Environment Variable Misconfiguration**
  - Missing or incorrect API keys and service URLs caused runtime errors on first start
- **Inconsistent Service Startup Order**
  - Some services failed when their dependencies were not started in the correct sequence
- **Repository Clone Authentication**
  - The git clone used by the coverage agent rejected the token scheme accepted by the GitHub REST API, so every authenticated clone failed until the credential handling was changed

Resolutions and Mitigation

The identified issues were resolved using the following approaches:

- **Disk Space**
  - Removed unused images and build cache with a system prune, freeing approximately 6 GB, and re-ran the image pulls
- **Database SSL Requirement**
  - Added the required SSL mode parameter to the affected connection string and verified every other service's connection string carried the same setting
- **CORS Errors**
  - Added the Vercel client domain to the gateway's allowed origins, including preflight and credentialed requests, and redeployed the gateway
- **Streaming Through the Proxy**
  - Enabled WebSocket upgrade handling and disabled response buffering for streamed log endpoints at both the proxy and gateway
- **Integration Issues**
  - Standardized API request/response formats (JSON) and verified and corrected service URLs and route prefixes
- **Environment Variables**

    - Created a centralized configuration reference and verified all required keys and service URLs before startup
- **Service Startup Issues**
    - Defined a controlled startup sequence: database → Authentication Service → gateway → research services → repair agent → reverse proxy → web client, gating each stage on a health check
- **Repository Clone Authentication**
    - Switched the clone credential to the scheme expected by git over HTTPS, passed as a short-lived header rather than embedded in the URL, while REST calls were left unchanged

Impact Analysis of Issues

Operational Impact

- Initial deployment was delayed by disk space, certificate, CORS, streaming and credential-scope configuration work
- End-to-end workflows spanning more than one service were temporarily unavailable until the gateway and proxy configuration was correct

System Impact

- Moderate impact during the setup phase, concentrated in the edge, gateway and external-integration layers
- Minimal impact after stabilization; the one-database-per-service design meant that no issue in one service could corrupt another service's data

User Impact

- Limited, as deployment was conducted in a controlled development/UAT environment with no external users

Residual Risk

- Risk of configuration mismatch on redeployment if environment variables, ports and credential scopes are not maintained in the documented reference
- Single-host dependency: the cloud instance is a single point of failure for every backend service
- The coverage agent runs an analysed repository's own commands on the backend host, mitigated only by the access-control boundary

Monitoring, Performance Metrics, and Feedback

Monitoring Methods

- Aggregate health endpoint reporting the live status of every service behind the gateway
- Reverse proxy access and error logs at the edge
- Per-container logs for each service for error detection and debugging
- In-product live transparency: the Agent Explorer stream, the execution terminal and the coverage log panel, each surfacing failures to the operator in real time
- Manual verification of role-based workflows (Business Analyst, QA Engineer, Developer, QA Lead)

Performance Metrics (Observed)

- Requirement Classification Accuracy: ~94%
- Generated Test Executability: ~91% (grounded)
- Root-Cause Classification Accuracy: ~89%
- Unsafe Repository Modification Rate: 0%
- RTM Update Latency: ~150 ms

- System Availability: High after stabilization

Monitoring Results

- All core services were responsive and stable after configuration fixes
- API communication between the gateway and all backend services was successful
- Core workflows were fully functional for testing and demonstration

User / Stakeholder Feedback

- Practitioners rated generated user stories as usable and well grounded in meeting evidence
- QA users found grounded test generation reduced the manual effort of writing and fixing selectors
- Developers expressed greater trust in controlled, approval-gated self-healing than in fully automatic repair
- Live RTM status and ranked coverage gaps were considered useful for QA leads
- Suggestions were made to:
    - Improve UI/UX consistency across the four service interfaces
    - Add centralized monitoring and alerting
    - Extend support to more languages and CI providers

4. Post-Deployment and Lessons Learned

Validation and Testing Results After Deployment

After deployment, the NextGen QA system was validated through multiple testing scenarios to ensure that all components function correctly as an integrated solution.

Validation Results:

- The public HTTPS endpoint was reachable with a valid certificate and every service reported healthy at the aggregate health endpoint
- A single Authentication Service token was accepted by the gateway and every protected service without a second login
- No service accessed another service's database; all cross-service data arrived through defined APIs
- End-to-end workflows were successfully executed:
    - Meeting audio transcribed, utterances classified, conflicts resolved and a validated user story generated with a quality score
    - Gherkin generated and approved, element map captured, multi-framework code generated and executed on a clean CI runner with a report
    - Failure classified with confidence, approval withheld producing no GitHub change, approval granted producing exactly one branch, one commit and one draft pull request
    - Tests inventoried and scored, coverage obtained by real execution, gaps ranked by risk, RTM status updated and a recommendation routed back to test generation
- Cross-service communication between the gateway and all backend services was stable after resolving configuration issues
- The system demonstrated functional completeness (~90–95%), meeting the evaluation requirements

Testing Results:

- Unit, integration and UAT testing confirmed that all major features work as expected in the deployed environment
- Grounded test generation improved first-run executability compared with ungrounded generation

- The Self-Healing Service correctly routed every failure category to its recovery action and never modified a repository without approval
- System response times and performance were within acceptable limits for interactive use

Documentation Updates and Training / Support Activities

Following deployment, several documentation and support activities were carried out to ensure smooth system usage and future maintainability.

Documentation Updates:

- Updated deployment documentation to reflect:
  - Service ports, gateway route prefixes and the public endpoint
  - Environment variables, their owning service and configuration setup
  - Correct service startup sequence and the health-check gate at each stage
- Documented common issues and troubleshooting steps (disk space, database SSL, CORS, certificate issuance, streaming, port binding and credential scope)
- Created a deployment checklist and configuration reference guide for team use
- Updated the system architecture diagram and per-service workflow descriptions based on the actual deployed implementation

Training and Support Activities:

- Provided guidance to team members on:
  - System startup and deployment process, and redeploying a single service without disturbing the others
  - Testing workflows for each role (Business Analyst, QA Engineer, Developer, QA Lead)
  - Debugging common errors and interpreting container logs
- Demonstrated how to:
  - Verify service health
  - Test API endpoints
  - Run end-to-end system workflows and read the live log panels
- Shared best practices for:
  - Managing environment variables
  - Maintaining consistency across services and avoiding configuration-related issues

Key Lessons Learned

- Integrating four independently developed services requires a single owner for the shared surfaces such as gateway routes, token format, database boundaries and error formats
- One database per service with API-only communication cost more integration work upfront but meant no failure in one service could corrupt another's data
- Real-time channels are the most fragile part of a proxied deployment; WebSocket and streamed logs behave correctly locally and fail silently behind a reverse proxy, so they must be tested end to end
- Safety must be enforced structurally rather than by intent: the confidence gate, deterministic policy, allow-list, approval gate and draft-only pull requests are what kept unsafe modifications at zero
- Grounding generated tests in the real application improves executability more than prompt tuning does
- Early validation of network connectivity, DNS, certificate issuance and CORS settings reduces deployment issues

- Environment variables and port assignments should be treated as critical system components and documented like code
- Incremental testing (unit → integration → UAT) improves system stability
- Clear documentation and communication within the team are essential for successful deployment

Recommended Future Improvements

- Move from single-host Docker Compose to a managed orchestrator with per-service scaling, rolling updates and health-gated deployment
- Sandbox the coverage agent in a disposable container or virtual machine so untrusted repository code is never executed directly on the backend host
- Set up centralized monitoring with aggregated logs, per-service metrics, tracing across the gateway and alerting on health regressions
- Move secrets into a managed secret store with rotation and per-service scoping
- Build a CI/CD pipeline for the platform itself with automated tests, image builds, staged rollout and one-command rollback
- Add caching, batching and rate-limit handling around AI model calls to reduce cost and variance
- Extend the platform to multilingual meetings and additional CI providers beyond GitHub Actions
- Add model monitoring and retraining for the AI services to keep accuracy high over time

Application Demo Video

https://mysliit-my.sharepoint.com/:f:/g/personal/it22251114_my_sliit_lk/IgA2YUmpUW0xTbylM2wywNydAaih-wrTBl7pwueRTiWyWh0?e=jVtBbb